

TECHNICAL BLUEPRINT · MULTI-AGENT AUTONOMY

Warehouse Autonomy Stack

A decentralized, vision-and-inertial-driven navigation and task-allocation architecture for a collaborating UAV and ground rover.

● UAV Agent — Stereo + IMU + 1D Lidar

● Ground Rover Agent

● Decentralized Swarm Link

Project: depth_perception_engine | Document type: Architecture Blueprint (target design, see Implementation Status for what is built today) | Compute target: NVIDIA Jetson Orin Nano | Flight controller: Pixhawk (PX4/ArduPilot, MAVLink)

Table of Contents

- 1. Executive Summary
- 2. Project Vision & Use Case
- 3. System-Level Architecture
- 4. Hardware Stack
- 5. Software Architecture — Per-Agent Pipeline
- 6. Multi-Agent Coordination Layer
- 7. End-to-End Workflow
- 8. Safety & Failsafe Architecture
- 9. Current Implementation Status vs. Blueprint
- 10. Technology Choices Summary
- 11. Phased Build Roadmap
- 12. Appendix — Deferred / Out-of-Scope Items

1. Executive Summary

This document specifies the target architecture for a decentralized, two-agent autonomous system — one aerial (UAV) and one ground (rover) — collaborating on warehouse automation tasks such as inventory scanning and item retrieval. Both agents share an identical perception-to-command software pipeline built from stereo vision, a dedicated time-synced IMU, and a single-point (1D) lidar for range aiding, fused into a Visual-Inertial state estimate (VIO / VI-SLAM). Navigation is purely geometric — no onboard object detection/classification is used, by deliberate design, to preserve compute and power budget on the target edge-compute platform (NVIDIA Jetson Orin Nano) for higher-priority workloads.

The two agents coordinate without a central fleet manager: tasks are announced, bid on, and assigned through a decentralized, capability-aware negotiation protocol carried over a mesh communication link. Each agent's own safety supervisor has final authority over any command sent to its flight controller (Pixhawk) or motor controller — including commands eventually produced by a learned (NN-based) local planner, which is treated as an optional, sandboxed contributor rather than a trusted authority.

Document status: this is a forward-looking architecture blueprint. Section 9 distinguishes precisely between what exists in the current codebase today (single-UAV, stereo-only reactive obstacle avoidance) and what is planned. Treat all other sections as target design, not as-built description.

2. Project Vision & Use Case

2.1 Problem framing

Most deployed warehouse robot fleets (e.g. Amazon Robotics, Locus Robotics, Berkshire Grey-class systems) use a *centralized* fleet manager directing a largely homogeneous ground fleet. This project targets a harder and more general problem: **heterogeneous, decentralized coordination** between an aerial agent and a ground agent, each with different capabilities, negotiating task ownership among themselves rather than being told what to do by a central broker.

2.2 Capability asymmetry — why two agent types

Capability	UAV	Ground Rover
Coverage speed	Fast, unobstructed by floor-level clutter and aisles	Slower, constrained to drivable floor space
Vantage point	Elevated / aerial view — good for wide-area scanning	Ground-level — good for under/behind-obstacle visibility
Payload	Limited (battery/lift constrained)	Higher payload capacity
Endurance	Shorter flight time	Longer operating duration
Position reliability	Drift-prone (inertial/visual only, no wheel reference)	More reliable (wheel odometry available as an additional reference)

Because no single agent dominates on every axis, task allocation is genuinely a function of capability and current state (location, battery, current task), not a fixed rule — which is why the allocation mechanism (Section 6) is a negotiation, not a lookup table.

2.3 Representative task

Example task cycle: a "scan aisle 4 and retrieve item X" task is announced on the shared task queue. The UAV, closer and with a clear flight path, bids to perform the aerial scan; the rover, positioned nearer the target shelf, bids to perform retrieval. Both subtasks execute concurrently, each agent running its own full perception-to-command loop independently.

3. System-Level Architecture

At the top level, the system is two independent, structurally identical agent stacks, joined only by a thin coordination layer and a decentralized comm link. Neither agent depends on the other to perceive, localize, or avoid obstacles — coordination affects only *which task* an agent works on, never how it perceives or moves.

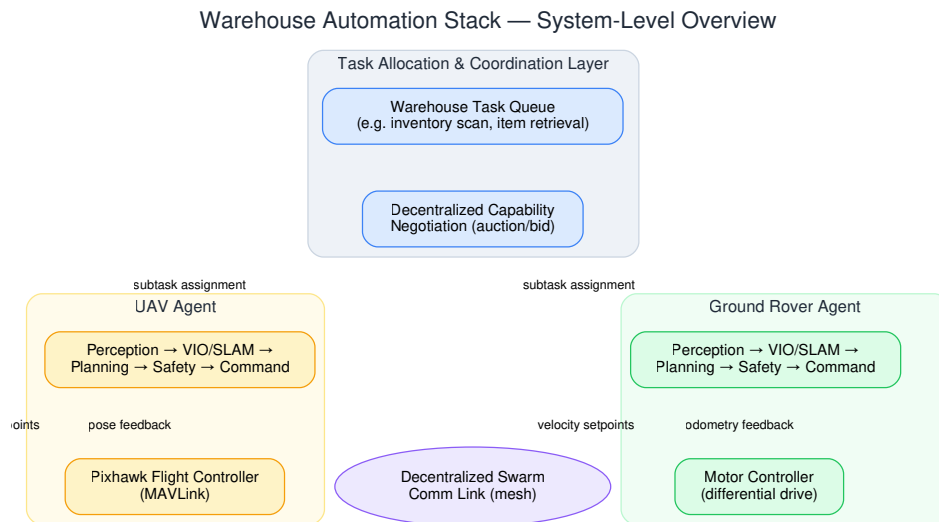


Figure 1 — System-level overview: two independently-operating agents joined by a decentralized coordination layer.

Design principle: the coordination layer is strictly an overlay. If the comm link is lost, each agent should be able to fall back to a sane default (e.g. complete current subtask, then hold) rather than depend on the link for basic safety.

4. Hardware Stack

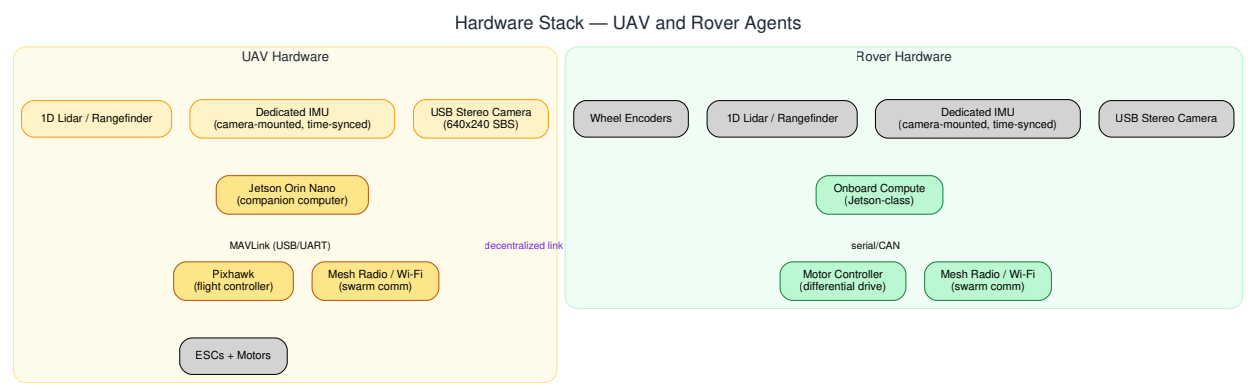


Figure 2 — Hardware stack for both agents, joined by the decentralized mesh link.

4.1 UAV hardware

Component	Role
USB stereo camera (640×240 side-by-side)	Primary geometric sensing — disparity/depth source
Dedicated IMU, rigidly mounted to camera, hardware/timestamp-synced	Inertial reference for VIO; deliberately separate from Pixhawk's own IMU so vision-inertial fusion isn't subject to MAVLink latency/jitter
1D lidar / single-point rangefinder	Range-aiding input to VIO — corrects scale/altitude drift that stereo+IMU alone is prone to over featureless surfaces
NVIDIA Jetson Orin Nano	Companion computer — runs the entire perception/VIO/planning/safety pipeline
Pixhawk (PX4 or ArduPilot)	Flight controller — attitude/rate control, motor mixing, hard failsafes; receives velocity setpoints via MAVLink OFFBOARD mode
Mesh radio / Wi-Fi link	Decentralized swarm communication with the rover

4.2 Rover hardware

Component	Role
USB stereo camera, dedicated synced IMU, 1D lidar	Same sensing suite as the UAV — identical perception/VIO pipeline reused without modification
Wheel encoders	Additional odometry reference unavailable to the UAV — improves rover pose reliability relative to the aerial agent
Onboard compute (Jetson-class)	Runs the same software stack as the UAV

Component	Role
Motor controller (differential drive)	Receives vehicle-agnostic velocity commands translated into wheel speeds
Mesh radio / Wi-Fi link	Decentralized swarm communication with the UAV

Note on the 1D lidar: a single-point rangefinder contributes a range/altitude constraint to the state estimator. It does not perform scanning and therefore does not, by itself, support mapping — that distinction matters because it means this system is VIO/VI-SLAM with range aiding, not "lidar SLAM" in the conventional (scanning-lidar) sense.

5. Software Architecture — Per-Agent Pipeline

Both agents run the identical pipeline below; only the final platform adapter differs (Pixhawk/MAVLink for the UAV, a differential-drive motor controller for the rover). This identical-pipeline property is what makes the stack reusable across agent types without duplicated logic.

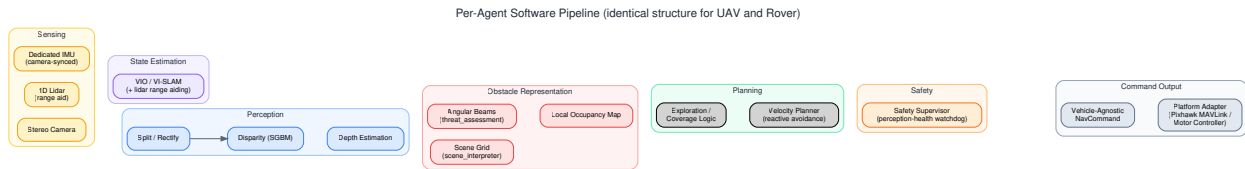


Figure 3 — Per-agent software pipeline, sensing through command output.

5.1 Sensing

Raw stereo frames, IMU samples, and 1D lidar range readings are acquired with consistent timestamping so that downstream fusion can associate the right samples with the right camera frame.

5.2 Perception

Stereo frames are split into left/right, rectified against calibration, and passed through SGBM-class stereo matching to produce a disparity map, then converted to metric depth.

5.3 State Estimation — VIO / VI-SLAM

IMU samples and stereo depth are fused into a pose estimate (position + heading) via Visual-Inertial Odometry, with the 1D lidar range reading fused in as a constraint that limits drift in scale/altitude. This pose estimate is what gives each agent a notion of "where am I relative to where I started" — something pure frame-by-frame depth cannot provide.

5.4 Obstacle Representation

Depth is reduced to two redundant geometric views used for avoidance: an angular beam decomposition (near/far per direction) and a coarse region grid (clear / obstacle / low-confidence per zone). A local occupancy map, built from depth plus the VIO pose over a short time window, gives the planner short-horizon memory beyond a single frame.

5.5 Planning

Two cooperating planning concerns: **exploration/coverage**, which biases movement toward unexplored space using the local map and current pose, and **reactive avoidance**, which is the final authority on immediate obstacle clearance regardless of where exploration wants to go.

5.6 Safety Supervisor

An independent watchdog, gated purely on perception/VIO health (stale frames, low confidence, sensor dropout) — never on pose or task state. It can override or clamp any candidate command, including one produced by an optional learned (NN) planner, before it is allowed to reach the command output stage. See Section 8 for the full failsafe hierarchy.

5.7 Command Output

The planner's output is expressed as a **vehicle-agnostic command** (linear + angular velocity), not a drone- or rover-specific format. A thin platform adapter converts this into Pixhawk MAVLink OFFBOARD setpoints for the UAV, or differential-drive wheel speeds for the rover. This is the abstraction that lets the same upstream pipeline serve both agent types unmodified.

6. Multi-Agent Coordination Layer

Task allocation uses a decentralized, auction/bundle-style negotiation (the same family as Consensus-Based Bundle Algorithm, CBBA) rather than a central broker: each agent computes its own cost to perform a given subtask from its current state and capability, bids are exchanged over the mesh link, and the lowest-cost capable agent wins — with no single point of failure or arbitration authority.

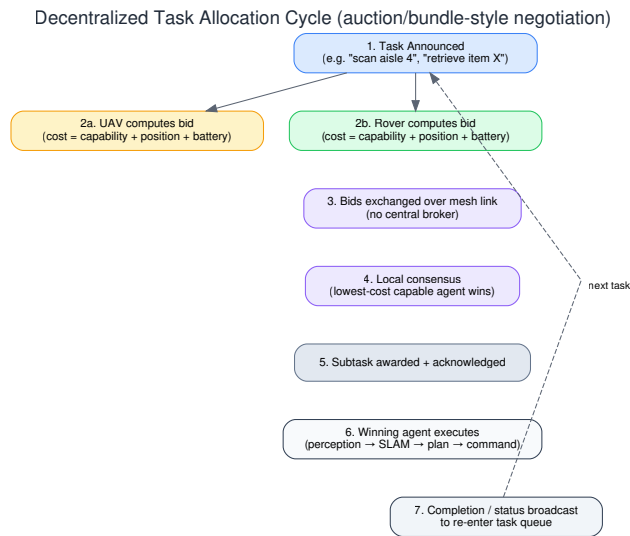


Figure 4 — One decentralized task allocation cycle.

6.1 Capability model

Each agent advertises a lightweight capability/cost function (e.g. distance to task, battery remaining, task-type affinity such as "aerial scan" vs. "ground retrieval") used by all agents to compute comparable bids without needing to share full internal state.

6.2 Comm link assumptions

The mesh link is assumed intermittent, not guaranteed-reliable — the negotiation protocol and each agent's local fallback behavior must tolerate dropped or delayed bid messages without deadlocking the task queue.

7. End-to-End Workflow

Walking one task cycle end-to-end, combining Sections 5 and 6:

1. **Task announced** on the shared queue (operator-issued or generated by a higher-level mission plan).
2. **Each agent bids** independently, using its own capability model and current pose/battery state.
3. **Bids are exchanged and resolved locally** by every agent reaching the same conclusion from the same shared information — no central arbiter.
4. **The winning agent executes** its own full per-agent pipeline (Section 5) — sensing through command output — entirely independently of the other agent.
5. **Safety supervision runs continuously** throughout execution, with authority to override the planner at any point regardless of task progress.
6. **Completion is broadcast**, returning the agent to availability for the next task.

8. Safety & Failsafe Architecture

Safety is layered, with each layer independent of and unaware of the layers above it — a fault in planning or perception should never require the flight/motor controller's own hard failsafes to know why, only that the command stream has gone stale or out of bounds.

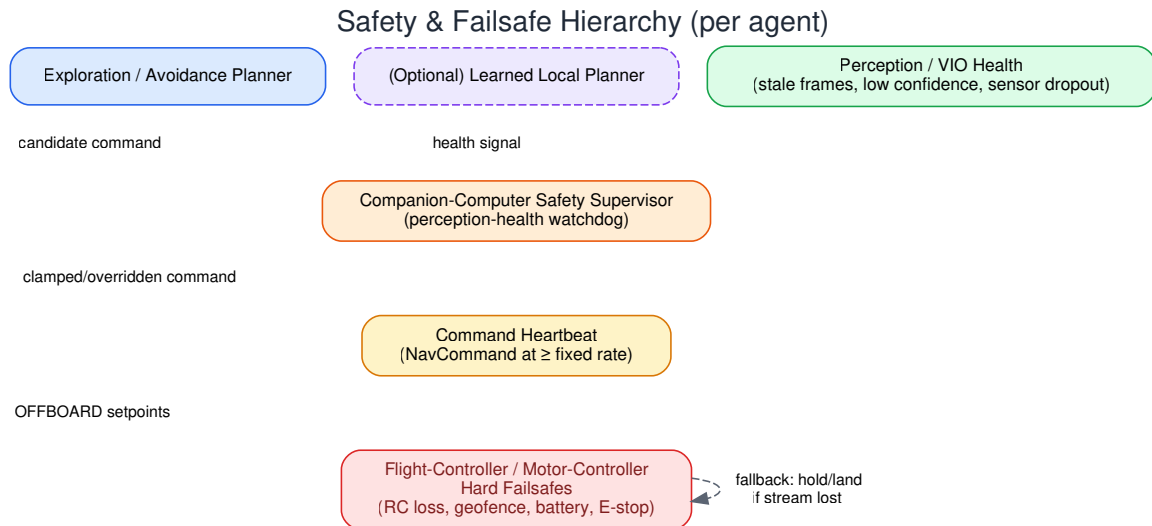


Figure 5 — Safety and failsafe hierarchy, per agent.

8.1 Command heartbeat

Following the standard PX4 OFFBOARD pattern, velocity setpoints must be streamed at a guaranteed minimum rate; if the stream stops (perception fault, software crash, comm loss), the flight/motor controller falls back to hold/land independent of any companion-computer logic.

8.2 NN sandboxing

Per the industry pattern described in Section 5.6, any learned/NN-based local planner is treated as one more candidate-command source feeding the safety supervisor — never as a trusted path directly to the command output stage. This is deliberate: it allows experimenting with learned planning without changing the safety guarantees of the system.

9. Current Implementation Status vs. Blueprint

The table below maps every layer in this blueprint to its actual state in the `depth_perception_engine` codebase as of this document's writing.

Layer	Status	Existing file(s)	Notes
Capture / split / rectify	Built & wired	stereo/frame_splitter.py, stereo/rectification.py	Single UAV camera path, in active use
Stereo matching (disparity)	Built & wired	stereo/disparity_engine.py	CPU SGBM; not yet validated/ profiled on Orin Nano
Depth estimation	Built & wired	depth/depth_estimator.py, depth/distance_reader.py	—
Obstacle beams	Built & wired	fusion/threat_assessment.py	20-beam angular decomposition
Scene grid	Built & wired	perception/scene_interpreter.py, region_analyzer.py, region_types.py	3×3 region classification
Reactive velocity planner	Built & wired	navigation/velocity_planner.py	Outputs forward_mps / yaw_rate_dps today (drone-shaped, not yet vehicle-agnostic)
Telemetry / run logging	Built & wired	telemetry/run_logger.py	—
Visualization (debug)	Built & wired	visualization/overlay_renderer.py	Desk-test only, not flight-critical
Object detection (YOLO)	Built, unwired — deprioritized	detection/yolo_detector.py, fusion/object_depth_fusion.py, fusion/region_classifier.py	Old roadmap; deliberately excluded from current blueprint (compute budget reserved on Orin Nano)
Object tracking / detection manager	Empty stub	detection/detection_manager.py, detection/object_tracker.py	0 bytes — not started
Dashboard / depth viewer	Empty stub	visualization/dashboard.py, visualization/depth_viewer.py	0 bytes — not started
Dedicated IMU ingestion	Planned	—	Hardware not yet connected

Layer	Status	Existing file(s)	Notes
VIO / VI-SLAM fusion	Planned	—	Depends on IMU ingestion
1D lidar range fusion	Planned	—	Hardware not yet connected
Local occupancy map	Planned	—	Depends on VIO pose
Exploration/ coverage logic	Planned	—	Depends on local map + pose
Vehicle-agnostic command schema	Planned	—	Current planner output is drone-shaped
Safety supervisor (formal module)	Partially implicit	main.py (stale-frame / too-close handling)	Logic exists inline in main.py; not yet a standalone, reusable module
Pixhawk MAVLink bridge	Planned	—	No FC integration exists yet; pipeline only logs/displays commands today
Rover stack	Planned	—	Not started; intended to reuse the UAV pipeline unmodified above the platform adapter
Decentralized task allocation	Planned	—	Not started

Built & wired = in the active main.py loop today. **Unwired** = code exists but nothing imports it. **Planned** = blueprint only, no code yet.

10. Technology Choices Summary

Concern	Approach / Candidate Tech	Rationale
Stereo matching	OpenCV SGBM (CPU today; CUDA SGBM candidate for Orin Nano)	Already implemented; CUDA port is the main open performance question for onboard deployment
VIO / VI-SLAM	Tightly-coupled stereo-inertial filter or optimization-based backend (e.g. an OpenVINS/VINS-Fusion-class approach), with 1D lidar fused as a range constraint	Avoids dependence on GPS or on Pixhawk's own drift-prone indoor dead reckoning
Command interface to flight controller	MAVLink OFFBOARD mode, vision pose pushed back via VISION_POSITION_ESTIMATE/ODOMETRY	Standard, well-tested pattern for vision-driven indoor PX4 flight
Task allocation	Decentralized auction/bundle negotiation (CBBA-family)	No single point of failure; matches the heterogeneous-capability requirement
Comm link	Mesh radio / Wi-Fi ad-hoc	Must tolerate intermittent connectivity without deadlocking task allocation
Onboard compute	NVIDIA Jetson Orin Nano	Compute/power budget is the reason object detection (YOLO) is excluded from this blueprint
Object detection / semantic labeling	Explicitly excluded	Reserves compute/power for VIO/SLAM and any future onboard NN planner

11. Phased Build Roadmap

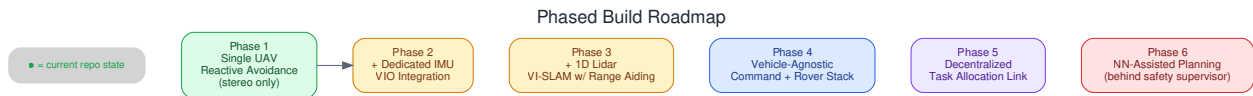


Figure 6 — Phased roadmap from current state to full multi-agent system.



12. Appendix — Deferred / Out-of-Scope Items

Object detection / semantic labeling (YOLO): deliberately excluded from this blueprint. Reasoning: obstacle avoidance only requires geometry, not object identity, and the compute/power this would consume on the Jetson Orin Nano is reserved for VIO/SLAM and any future onboard NN planner. The existing `detection/` and `fusion/object_depth_fusion.py`, `fusion/region_classifier.py` modules remain in the codebase as built-but-unwired legacy work from an earlier roadmap; revisit only if this constraint changes.

Full SLAM with loop closure / persistent mapping: not adopted by default. The blueprint specifies lightweight, local (rolling-window) mapping sufficient for single-room exploration and avoidance. Full pose-graph SLAM with loop closure is heavier and only justified if a future mission layer genuinely requires a persistent, revisitable map — to be revisited if that need materializes.

End-to-end NN control (sensor input directly to motor output): not adopted. Industry practice — and this blueprint — keeps any NN strictly as one candidate-command source behind an independent, non-learned safety supervisor (Section 8.2), never as a trusted direct path to actuation.